

Competitive Programming

Section 72 Editorial: Master+ Flow And Convex Optimization Mixed Contest

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Exam Room Bounds	4
2.1	How to find it	4
2.2	Correctness	4
2.3	Complexity and tests	4
2.4	C++	4
2.5	Python	6
3	B. Night Delivery	9
3.1	How to find it	9
3.2	Correctness	9
3.3	Complexity and tests	9
3.4	C++	9
3.5	Python	11
4	C. Team Quota Profit	14
4.1	How to find it	14
4.2	Correctness	14
4.3	Complexity and tests	14
4.4	C++	14
4.5	Python	17
5	D. Monotone Signal	20
5.1	How to find it	20
5.2	Correctness	20
5.3	Complexity and tests	20
5.4	C++	20
5.5	Python	21
6	E. Exact Discount Tree	22
6.1	How to find it	22
6.2	Correctness	22
6.3	Complexity and tests	22
6.4	C++	22
6.5	Python	24
7	F. Congested Team Assignment	26
7.1	How to find it	26
7.2	Correctness	26
7.3	Complexity and tests	26
7.4	C++	26
7.5	Python	29

1 How to use this editorial

For each problem, identify the reduction before reading the code. The proof paragraph names the invariant that should have appeared in your own derivation.

2 A. Exam Room Bounds

Create source-to-group edges fixed at each demand, allowed unit group-to-room edges, room-to-sink bounded edges, and a sink-to-source edge fixed at total demand. This is a bounded circulation, so the balance reduction decides feasibility.

2.1 How to find it

Rows and columns are two sides of a bipartite unit-flow model. Row demands are fixed source-edge flows, room limits are lower/upper sink-edge bounds, and an allowed pair is a unit middle edge. Fixing the return edge to the total demand turns the whole construction into a circulation.

2.2 Correctness

A valid selection sends one unit through exactly each selected pair; row and column constraints become the stated edge bounds, so it produces a feasible circulation. Conversely, integral max flow makes every middle edge zero or one. Reading its unit edges gives distinct allowed choices, with exact row degrees and bounded column degrees. The bounded-circulation equivalence then decides the instance exactly.

2.3 Complexity and tests

The graph has $R+C+4$ vertices and $E+R+C+O(1)$ edges. Test duplicate-looking constraints, a room minimum unreachable from enough groups, zero row demands, and feasible totals whose allowed-pair structure is nevertheless impossible.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

struct Dinic {
    struct Edge {
        int to, rev;
        int64 cap;
    };
    vector<vector<Edge>> g;
```

```

vector<int> level, it;
Dinic(int n) : g(n), level(n), it(n) {}
void add_edge(int u, int v, int64 cap) {
    Edge a{v, (int)g[v].size(), cap};
    Edge b{u, (int)g[u].size(), 0};
    g[u].push_back(a);
    g[v].push_back(b);
}
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    level[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto &e : g[u])
            if (e.cap && level[e.to] < 0)
                level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] >= 0;
}
int64 dfs(int u, int t, int64 pushed) {
    if (u == t)
        return pushed;
    for (int &i = it[u]; i < (int)g[u].size(); ++i) {
        Edge &e = g[u][i];
        if (!e.cap || level[e.to] != level[u] + 1)
            continue;
        int64 take = dfs(e.to, t, min(pushed, e.cap));
        if (take) {
            e.cap -= take;
            g[e.to][e.rev].cap += take;
            return take;
        }
    }
    return 0;
}
int64 flow(int s, int t) {
    int64 ans = 0, pushed;
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while ((pushed = dfs(s, t, (1LL << 62))))
            ans += pushed;
    }
    return ans;
}
};

int main() {
    ios::sync_with_stdio(false);

```

```

cin.tie(nullptr);
int rows, cols, m;
if (!(cin >> rows >> cols >> m))
    return 0;
int n = rows + cols + 2;
int source = rows + cols, sink = source + 1, ss = n, tt = n + 1;
Dinic dinic(n + 2);
vector<int64> balance(n);
int64 total = 0;
auto bounded = [&](int u, int v, int64 low, int64 high) {
    dinic.add_edge(u, v, high - low);
    balance[u] -= low;
    balance[v] += low;
};
for (int i = 0; i < rows; ++i) {
    int64 demand;
    cin >> demand;
    bounded(source, i, demand, demand);
    total += demand;
}
for (int j = 0; j < cols; ++j) {
    int64 low, high;
    cin >> low >> high;
    bounded(rows + j, sink, low, high);
}
for (int i = 0, r, c; i < m; ++i) {
    cin >> r >> c;
    bounded(--r, rows + --c, 0, 1);
}
bounded(sink, source, total, total);
int64 need = 0;
for (int v = 0; v < n; ++v) {
    if (balance[v] > 0)
        dinic.add_edge(ss, v, balance[v]), need += balance[v];
    if (balance[v] < 0)
        dinic.add_edge(v, tt, -balance[v]);
}
cout << (dinic.flow(ss, tt) == need ? "YES\n" : "NO\n");
}

```

2.5 Python

```

import collections
import sys

class Dinic:
    def __init__(self, n):

```

```

self.g = [[] for _ in range(n)]

def add_edge(self, u, v, cap):
    self.g[u].append([v, cap, len(self.g[v])])
    self.g[v].append([u, 0, len(self.g[u]) - 1])

def flow(self, s, t):
    answer = 0
    while True:
        level = [-1] * len(self.g)
        level[s] = 0
        q = collections.deque([s])
        while q:
            u = q.popleft()
            for v, cap, _ in self.g[u]:
                if cap and level[v] < 0:
                    level[v] = level[u] + 1
                    q.append(v)
        if level[t] < 0:
            return answer
        it = [0] * len(self.g)

    def dfs(u, pushed):
        if u == t:
            return pushed
        while it[u] < len(self.g[u]):
            e = self.g[u][it[u]]
            v, cap, rev = e
            if cap and level[v] == level[u] + 1:
                take = dfs(v, min(pushed, cap))
                if take:
                    e[1] -= take
                    self.g[v][rev][1] += take
                    return take
            it[u] += 1
        return 0

    while pushed := dfs(s, 10**30):
        answer += pushed

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    rows, cols, m = data[:3]
    at = 3
    demands = data[at : at + rows]
    at += rows
    bounds = []
    for _ in range(cols):

```

```

        bounds.append(tuple(data[at : at + 2]))
        at += 2
n = rows + cols + 2
source, sink, ss, tt = rows + cols, rows + cols + 1, n, n + 1
dinic = Dinic(n + 2)
balance = [0] * n

def bounded(u, v, low, high):
    dinic.add_edge(u, v, high - low)
    balance[u] -= low
    balance[v] += low

total = sum(demands)
for r, demand in enumerate(demands):
    bounded(source, r, demand, demand)
for c, (low, high) in enumerate(bounds):
    bounded(rows + c, sink, low, high)
for _ in range(m):
    r, c = data[at : at + 2]
    at += 2
    bounded(r - 1, rows + c - 1, 0, 1)
bounded(sink, source, total, total)
need = 0
for v, value in enumerate(balance):
    if value > 0:
        dinic.add_edge(ss, v, value)
        need += value
    elif value < 0:
        dinic.add_edge(v, tt, -value)
print("YES" if dinic.flow(ss, tt) == need else "NO")

if __name__ == "__main__":
    main()

```

3 B. Night Delivery

A residual reverse edge permits replacing earlier choices. Initial Bellman–Ford potentials make every reachable reduced cost nonnegative; Dijkstra then finds each cheapest augmentation. Updating reached potentials by their Dijkstra distances preserves feasibility. Stop after exactly F units or report impossibility.

3.1 How to find it

A greedy cheapest original path fails because a later unit may need to undo an earlier route. That observation asks for a residual graph: every used edge creates a reverse edge of opposite cost. We still want shortest augmenting paths, so potentials are introduced solely to make Dijkstra legal.

3.2 Correctness

Feasible potentials make all reduced residual costs nonnegative, and path reweighting changes every s – t path cost by the same constant. Dijkstra therefore chooses a truly cheapest residual augmenting path. Updating reached potentials with shortest reduced distances preserves non-negative reduced costs by the triangle inequality. The standard residual optimality criterion then shows inductively that after each augmentation the current flow has minimum cost among flows of its value. Hence the state after F units is the required optimum.

3.3 Complexity and tests

With integral capacities, at most F augmentations are needed here, for $O(F E \log V)$ after the initial shortest-distance computation. Test negative input edges, a case where a reverse residual edge changes the first choice, parallel routes, zero requested flow, and insufficient capacity.

3.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MCF {
    struct Edge {
        int to, rev, cap, initial;
```

```

    int64 cost;
};
vector<vector<Edge>> g;
MCF(int n) : g(n) {}
pair<int, int> add_edge(int u, int v, int cap, int64 cost) {
    int id = g[u].size();
    g[u].push_back({v, (int)g[v].size(), cap, cap, cost});
    g[v].push_back({u, id, 0, 0, -cost});
    return {u, id};
}
pair<int, int64> flow(int s, int t, int limit) {
    int n = g.size(), sent = 0;
    int64 cost = 0;
    vector<int64> pot(n, INF), dist(n);
    pot[s] = 0;
    for (int rep = 0; rep < n; ++rep) {
        bool changed = false;
        for (int u = 0; u < n; ++u)
            if (pot[u] < INF)
                for (auto &e : g[u])
                    if (e.cap && pot[e.to] > pot[u] + e.cost)
                        pot[e.to] = pot[u] + e.cost, changed = true;
        if (!changed)
            break;
    }
    for (auto &x : pot)
        if (x == INF)
            x = 0;
    vector<int> pv(n), pe(n);
    while (sent < limit) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>>
            pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u])
                continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                int64 nd = d + e.cost + pot[u] - pot[e.to];
                if (e.cap && nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
        if (dist[t] == INF)
            break;
        for (int v = 0; v < n; ++v)

```

```

        if (dist[v] < INF)
            pot[v] += dist[v];
        int add = limit - sent;
        for (int v = t; v != s; v = pv[v])
            add = min(add, g[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            Edge &e = g[pv[v]][pe[v]];
            cost += (int64)add * e.cost;
            e.cap -= add;
            g[v][e.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, need;
    if (!(cin >> n >> m >> need))
        return 0;
    int s, t;
    cin >> s >> t;
    --s;
    --t;
    MCF mcf(n);
    for (int i = 0, u, v, cap; i < m; ++i) {
        int64 cost;
        cin >> u >> v >> cap >> cost;
        mcf.add_edge(--u, --v, cap, cost);
    }
    auto [sent, cost] = mcf.flow(s, t, need);
    if (sent < need)
        cout << "IMPOSSIBLE\n";
    else
        cout << cost << '\n';
}

```

3.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):

```

```

self.g = [[] for _ in range(n)]

def add_edge(self, u, v, cap, cost):
    ref = (u, len(self.g[u]))
    self.g[u].append([v, cap, cost, len(self.g[v]), cap])
    self.g[v].append([u, 0, -cost, len(self.g[u]) - 1, 0])
    return ref

def flow(self, s, t, limit):
    n = len(self.g)
    inf = 10**40
    potential = [inf] * n
    potential[s] = 0
    for _ in range(n):
        changed = False
        for u in range(n):
            if potential[u] == inf:
                continue
            for v, cap, cost, _, _ in self.g[u]:
                if cap and potential[v] > potential[u] + cost:
                    potential[v] = potential[u] + cost
                    changed = True
        if not changed:
            break
    potential = [0 if x == inf else x for x in potential]
    sent = total = 0
    while sent < limit:
        dist = [inf] * n
        prev = [None] * n
        dist[s] = 0
        pq = [(0, s)]
        while pq:
            d, u = heapq.heappop(pq)
            if d != dist[u]:
                continue
            for i, (v, cap, cost, _, _) in enumerate(self.g[u]):
                nd = d + cost + potential[u] - potential[v]
                if cap and nd < dist[v]:
                    dist[v] = nd
                    prev[v] = (u, i)
                    heapq.heappush(pq, (nd, v))
        if dist[t] == inf:
            break
        for v in range(n):
            if dist[v] < inf:
                potential[v] += dist[v]
        add = limit - sent
        v = t
        while v != s:
            u, i = prev[v]
            add = min(add, self.g[u][i][1])

```

```

        v = u
    v = t
    while v != s:
        u, i = prev[v]
        e = self.g[u][i]
        total += add * e[2]
        e[1] -= add
        self.g[v][e[3]][1] += add
        v = u
    sent += add
    return sent, total

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, need, s, t = data[:5]
    mcf = MCF(n)
    at = 5
    for _ in range(m):
        u, v, cap, cost = data[at : at + 4]
        at += 4
        mcf.add_edge(u - 1, v - 1, cap, cost)
    sent, cost = mcf.flow(s - 1, t - 1, need)
    print(cost if sent == need else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

4 C. Team Quota Profit

Send one unit through each worker. Project-to-sink capacity enforces upper quotas. Give the first `low[j]` parallel slots a reward `-BIG`, where `BIG` exceeds every possible difference in real profit, and verify all rewarded slots are saturated. The remaining min-cost objective is precisely negative total profit.

4.1 How to find it

“Every worker exactly once” naturally means one unit leaving every worker; eligibility edges encode the choices and their costs are negative profits. Upper quotas are capacities. To use a plain min-cost-flow implementation for lower quotas, turn the first `low[j]` project slots into lexicographically mandatory slots with reward `-BIG`.

4.2 Correctness

Every integral flow of value `n` decomposes into one eligible project choice per worker, and every assignment yields such a flow. Project capacities enforce the upper bounds. If a quota-feasible assignment exists, it fills all mandatory slots. Because `BIG` exceeds the maximum possible difference between two total profits, every minimum-cost flow first maximizes the number of mandatory slots filled and only then maximizes real profit. Verifying their saturation rejects exactly the infeasible instances; removing their constant reward recovers the true optimum.

4.3 Complexity and tests

There are $O(n+p)$ vertices and $O(n+m+p)$ edges. Successive shortest paths take $O(n \cdot E \cdot \log V)$. Test a worker with no choices, tight quotas, infeasible lower quotas despite sufficient total capacity, and an optimum using a lower-profit edge to satisfy a project minimum.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MCF {
    struct Edge {
        int to, rev, cap, initial;
```

```

    int64 cost;
};
vector<vector<Edge>> g;
MCF(int n) : g(n) {}
pair<int, int> add_edge(int u, int v, int cap, int64 cost) {
    int id = g[u].size();
    g[u].push_back({v, (int)g[v].size(), cap, cap, cost});
    g[v].push_back({u, id, 0, 0, -cost});
    return {u, id};
}
pair<int, int64> flow(int s, int t, int limit) {
    int n = g.size(), sent = 0;
    int64 cost = 0;
    vector<int64> pot(n, INF), dist(n);
    pot[s] = 0;
    for (int rep = 0; rep < n; ++rep) {
        bool changed = false;
        for (int u = 0; u < n; ++u)
            if (pot[u] < INF)
                for (auto &e : g[u])
                    if (e.cap && pot[e.to] > pot[u] + e.cost)
                        pot[e.to] = pot[u] + e.cost, changed = true;
        if (!changed)
            break;
    }
    for (auto &x : pot)
        if (x == INF)
            x = 0;
    vector<int> pv(n), pe(n);
    while (sent < limit) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>>
            pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u])
                continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                int64 nd = d + e.cost + pot[u] - pot[e.to];
                if (e.cap && nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
        if (dist[t] == INF)
            break;
        for (int v = 0; v < n; ++v)

```

```

        if (dist[v] < INF)
            pot[v] += dist[v];
        int add = limit - sent;
        for (int v = t; v != s; v = pv[v])
            add = min(add, g[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            Edge &e = g[pv[v]][pe[v]];
            cost += (int64)add * e.cost;
            e.cap -= add;
            g[v][e.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p, m;
    if (!(cin >> n >> p >> m))
        return 0;
    vector<int> low(p), high(p);
    int required = 0;
    for (int j = 0; j < p; ++j)
        cin >> low[j] >> high[j], required += low[j];
    struct Choice {
        int w, j, profit;
    };
    vector<Choice> choices(m);
    int max_profit = 0;
    for (auto &[w, j, profit] : choices) {
        cin >> w >> j >> profit;
        --w;
        --j;
        max_profit = max(max_profit, profit);
    }
    int s = n + p, t = s + 1;
    MCF mcf(t + 1);
    for (int w = 0; w < n; ++w)
        mcf.add_edge(s, w, 1, 0);
    for (auto [w, j, profit] : choices)
        mcf.add_edge(w, n + j, 1, -profit);
    int64 big = (int64)(n + 1) * (max_profit + 1);
    vector<pair<int, int>> mandatory;
    for (int j = 0; j < p; ++j) {
        mandatory.push_back(mcf.add_edge(n + j, t, low[j], -big));
        mcf.add_edge(n + j, t, high[j] - low[j], 0);
    }
    auto [sent, cost] = mcf.flow(s, t, n);

```

```

bool ok = sent == n;
for (auto [u, id] : mandatory)
    ok &= mcf.g[u][id].cap == 0;
if (!ok)
    cout << "IMPOSSIBLE\n";
else
    cout << -cost - big * required << '\n';
}

```

4.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add_edge(self, u, v, cap, cost):
        ref = (u, len(self.g[u]))
        self.g[u].append([v, cap, cost, len(self.g[v]), cap])
        self.g[v].append([u, 0, -cost, len(self.g[u]) - 1, 0])
        return ref

    def flow(self, s, t, limit):
        n = len(self.g)
        inf = 10**40
        potential = [inf] * n
        potential[s] = 0
        for _ in range(n):
            changed = False
            for u in range(n):
                if potential[u] == inf:
                    continue
                for v, cap, cost, _, _ in self.g[u]:
                    if cap and potential[v] > potential[u] + cost:
                        potential[v] = potential[u] + cost
                        changed = True
            if not changed:
                break
        potential = [0 if x == inf else x for x in potential]
        sent = total = 0
        while sent < limit:
            dist = [inf] * n
            prev = [None] * n
            dist[s] = 0
            pq = [(0, s)]

```

```

        while pq:
            d, u = heapq.heappop(pq)
            if d != dist[u]:
                continue
            for i, (v, cap, cost, _, _) in enumerate(self.g[u]):
                nd = d + cost + potential[u] - potential[v]
                if cap and nd < dist[v]:
                    dist[v] = nd
                    prev[v] = (u, i)
                    heapq.heappush(pq, (nd, v))
        if dist[t] == inf:
            break
        for v in range(n):
            if dist[v] < inf:
                potential[v] += dist[v]
        add = limit - sent
        v = t
        while v != s:
            u, i = prev[v]
            add = min(add, self.g[u][i][1])
            v = u
        v = t
        while v != s:
            u, i = prev[v]
            e = self.g[u][i]
            total += add * e[2]
            e[1] -= add
            self.g[v][e[3]][1] += add
            v = u
        sent += add
    return sent, total

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, p, m = data[:3]
    at = 3
    low = []
    high = []
    for _ in range(p):
        lo, hi = data[at : at + 2]
        at += 2
        low.append(lo)
        high.append(hi)
    choices = []
    max_profit = 0
    for _ in range(m):
        w, j, profit = data[at : at + 3]
        at += 3

```

```

        choices.append((w - 1, j - 1, profit))
    max_profit = max(max_profit, profit)
s = n + p
t = s + 1
mcf = MCF(t + 1)
for w in range(n):
    mcf.add_edge(s, w, 1, 0)
for w, j, profit in choices:
    mcf.add_edge(w, n + j, 1, -profit)
big = (n + 1) * (max_profit + 1)
mandatory = []
for j in range(p):
    mandatory.append(mcf.add_edge(n + j, t, low[j], -big))
    mcf.add_edge(n + j, t, high[j] - low[j], 0)
sent, cost = mcf.flow(s, t, n)
ok = sent == n and all(mcf.g[u][i][1] == 0 for u, i in mandatory)
print(-cost - big * sum(low) if ok else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

5 D. Monotone Signal

The prefix optimum as a function of its final value is convex. Its derivative changes only at stored breakpoints. Adding $|x-a|$ inserts x twice; truncating choices that violate monotonicity removes the largest left breakpoint and charges its displacement. Thus the heap maintains the exact compressed derivative.

5.1 How to find it

Start from the quadratic DP “best prefix cost when the final repaired value is x .” Its value as a function of x is convex and piecewise linear. Storing all values is unnecessary: a convex piecewise-linear function is determined by the ordered positions where its derivative changes.

5.2 Correctness

The max-heap stores the left breakpoints of the prefix value function. Adding $|x-a[i]|$ introduces two unit slope changes at $a[i]$. The monotonicity transition takes a prefix minimum over legal previous final values; in the derivative representation this removes the largest left breakpoint. If that breakpoint is y , moving it to $a[i]$ increases the minimum by $y-a[i]$. These are exactly the two pushes, one pop, and cost update in the code. Induction over prefixes therefore proves that the accumulated minimum equals the isotonic L1 optimum.

5.3 Complexity and tests

Each item performs three heap operations, so time is $O(n \log n)$ and memory is $O(n)$. Test already sorted, strictly decreasing, repeated values, negative values, and prefixes whose optimal medians are not unique.

5.4 C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    priority_queue<long long> left;
```

```

long long answer = 0;
for (int i = 0; i < n; ++i) {
    long long x;
    cin >> x;
    left.push(x);
    left.push(x);
    answer += left.top() - x;
    left.pop();
}
cout << answer << '\n';
}

```

5.5 Python

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    heap = []
    answer = 0
    for x in data[1:]:
        heapq.heappush(heap, -x)
        heapq.heappush(heap, -x)
        answer += -heapq.heappop(heap) - x
    print(answer)

if __name__ == "__main__":
    main()

```

6 E. Exact Discount Tree

Price every red edge by `lambda`. Kruskal solves the penalized problem, and the red count is nonincreasing in `lambda`. Opposite tie rules give the extreme red counts among equally optimal trees. Matroid basis exchange fills the interval between them, so subtracting `lambda*k` recovers the exact-`k` objective.

6.1 How to find it

The difficult condition is the exact red count; everything else is an MST. Move that condition into the objective by charging `lambda` for each red edge. For a fixed charge the solver remains Kruskal, while increasing the charge can only decrease the red count of an optimum.

6.2 Correctness

Kruskal minimizes `weight + lambda*red`. Among equal modified weights, the two tie orders return the largest and smallest red counts of modified-optimal trees. The basis-exchange property of the graphic matroid connects optimal spanning trees by equal-cost exchanges, changing the red count one step at a time; consequently every count between those extremes is attainable at the same modified value. Binary search finds the last integer price whose red-preferring optimum still uses at least `k` reds. At that boundary `k` is in the optimal count interval, so subtracting `lambda*k` gives exactly the minimum original weight with `k` reds.

6.3 Complexity and tests

Each price evaluation sorts `m` edges, and $O(\log W)$ prices are tested, giving $O(m \log m \log W)$ here. Test impossible extreme counts, many equal modified weights, negative weights, all-one-color graphs, and cases with several MSTs.

6.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (x != p[x])
```

```

        x = p[x];
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b)
            return false;
        if (sz[a] < sz[b])
            swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

struct Edge {
    int u, v, red;
    int64 w;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, wanted;
    if (!(cin >> n >> m >> wanted))
        return 0;
    vector<Edge> edges(m);
    for (auto &e : edges) {
        char c;
        cin >> e.u >> e.v >> e.w >> c;
        --e.u;
        --e.v;
        e.red = c == 'R';
    }

    auto run = [&](int64 lambda, bool prefer_red) {
        vector<int> order(m);
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) {
            int64 a = edges[i].w + lambda * edges[i].red;
            int64 b = edges[j].w + lambda * edges[j].red;
            if (a != b)
                return a < b;
            return prefer_red ? edges[i].red > edges[j].red
                               : edges[i].red < edges[j].red;
        });
        DSU dsu(n);
        int used = 0, red = 0;
        int64 value = 0;
        for (int id : order)
            if (dsu.unite(edges[id].u, edges[id].v)) {
                ++used;
                red += edges[id].red;
            }
    };

```

```

        value += edges[id].w + lambda * edges[id].red;
    }
    return tuple<int, int, int64>{used, red, value};
};
auto [u1, min_red, z1] = run(400000000000000LL, false);
auto [u2, max_red, z2] = run(-400000000000000LL, true);
if (u1 != n - 1 || wanted < min_red || wanted > max_red) {
    cout << "IMPOSSIBLE\n";
    return 0;
}
int64 lo = -400000000000000LL, hi = 400000000000000LL;
while (lo < hi) {
    int64 mid = lo + (hi - lo + 1) / 2;
    auto [used, red, value] = run(mid, true);
    if (red >= wanted)
        lo = mid;
    else
        hi = mid - 1;
}
auto [used, red, modified] = run(lo, true);
cout << modified - lo * wanted << '\n';
}

```

6.5 Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = iter(data)
    n, m, wanted = int(next(it)), int(next(it)), int(next(it))
    edges = []
    for _ in range(m):
        u, v, w, color = (
            int(next(it)) - 1,
            int(next(it)) - 1,
            int(next(it)),
            next(it),
        )
        edges.append((u, v, w, color == b"R"))

    def run(lam, prefer_red):
        order = sorted(
            edges,
            key=lambda e: (e[2] + lam * e[3], -e[3] if prefer_red else e[3]),

```

```

)
parent = list(range(n))
size = [1] * n

def find(x):
    while x != parent[x]:
        x = parent[x]
    return x

used = red = value = 0
for u, v, w, is_red in order:
    u, v = find(u), find(v)
    if u == v:
        continue
    if size[u] < size[v]:
        u, v = v, u
    parent[v] = u
    size[u] += size[v]
    used += 1
    red += is_red
    value += w + lam * is_red
return used, red, value

bound = 400_000_000_000_000
used, minimum, _ = run(bound, False)
_, maximum, _ = run(-bound, True)
if used != n - 1 or not minimum <= wanted <= maximum:
    print("IMPOSSIBLE")
    return
lo, hi = -bound, bound
while lo < hi:
    mid = (lo + hi + 1) // 2
    if run(mid, True)[1] >= wanted:
        lo = mid
    else:
        hi = mid - 1
_, _, modified = run(lo, True)
print(modified - lo * wanted)

if __name__ == "__main__":
    main()

```

7 F. Congested Team Assignment

Worker-to-team edges contribute negative profit. Replace team cost $a \cdot x^2$ by x unit slots with marginal costs $a(2k-1)$. Reward the mandatory lower-quota slots by $-BIG$ and verify them afterward. A single exact min-cost flow now optimizes assignment and congestion together.

7.1 How to find it

This is assignment flow except the price of a team depends on its final load. Convexity makes the dependence separable: the k -th member costs the marginal $a[j](2k-1)$. Replace one nonlinear team edge by ordered unit slot edges, and use the lower-quota reward from the quota problem on its first slots.

7.2 Correctness

An integral value- n flow chooses one eligible team per specialist. Since a team's marginal slot costs are nondecreasing, any cheapest flow using x slots uses precisely the first x ; their sum is $a[j]x^2$. The $-BIG$ reward, chosen larger than the full real-objective range, makes saturation of every lower-quota slot lexicographically prior to profit and congestion. After the explicit saturation check, feasible flows and valid assignments correspond bijectively and have equal real objective, so the returned maximum is exact.

7.3 Complexity and tests

There are $O(n+p)$ vertices and $O(m+n+\text{sum high})$ edges, with at most n augmentations. Test zero congestion, tight project sizes, a quota forcing an unprofitable assignment, equal marginal slots, and infeasible eligibility.

7.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MCF {
    struct Edge {
        int to, rev, cap, initial;
        int64 cost;
```

```

};
vector<vector<Edge>> g;
MCF(int n) : g(n) {}
pair<int, int> add_edge(int u, int v, int cap, int64 cost) {
    int id = g[u].size();
    g[u].push_back({v, (int)g[v].size(), cap, cap, cost});
    g[v].push_back({u, id, 0, 0, -cost});
    return {u, id};
}
pair<int, int64> flow(int s, int t, int limit) {
    int n = g.size(), sent = 0;
    int64 cost = 0;
    vector<int64> pot(n, INF), dist(n);
    pot[s] = 0;
    for (int rep = 0; rep < n; ++rep) {
        bool changed = false;
        for (int u = 0; u < n; ++u)
            if (pot[u] < INF)
                for (auto &e : g[u])
                    if (e.cap && pot[e.to] > pot[u] + e.cost)
                        pot[e.to] = pot[u] + e.cost, changed = true;
        if (!changed)
            break;
    }
    for (auto &x : pot)
        if (x == INF)
            x = 0;
    vector<int> pv(n), pe(n);
    while (sent < limit) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>>
            pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u])
                continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                int64 nd = d + e.cost + pot[u] - pot[e.to];
                if (e.cap && nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
    }
    if (dist[t] == INF)
        break;
    for (int v = 0; v < n; ++v)
        if (dist[v] < INF)

```

```

        pot[v] += dist[v];
        int add = limit - sent;
        for (int v = t; v != s; v = pv[v])
            add = min(add, g[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            Edge &e = g[pv[v]][pe[v]];
            cost += (int64)add * e.cost;
            e.cap -= add;
            g[v][e.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p, m;
    if (!(cin >> n >> p >> m))
        return 0;
    vector<int> low(p), high(p), a(p);
    int required = 0, max_a = 0;
    for (int j = 0; j < p; ++j)
        cin >> low[j] >> high[j] >> a[j], required += low[j],
        max_a = max(max_a, a[j]);
    struct Choice {
        int w, j, profit;
    };
    vector<Choice> choices(m);
    int max_profit = 0;
    for (auto &[w, j, profit] : choices)
        cin >> w >> j >> profit, --w, --j, max_profit = max(max_profit, profit);
    int s = n + p, t = s + 1;
    MCF mcf(t + 1);
    for (int w = 0; w < n; ++w)
        mcf.add_edge(s, w, 1, 0);
    for (auto [w, j, profit] : choices)
        mcf.add_edge(w, n + j, 1, -profit);
    int64 big = (int64)(n + 1) * (max_profit + 2LL * max_a * n + 1);
    vector<pair<int, int>> mandatory;
    for (int j = 0; j < p; ++j)
        for (int k = 1; k <= high[j]; ++k) {
            int64 cost = (int64)a[j] * (2LL * k - 1);
            auto ref = mcf.add_edge(n + j, t, 1, cost - (k <= low[j] ? big : 0));
            if (k <= low[j])
                mandatory.push_back(ref);
        }
    auto [sent, cost] = mcf.flow(s, t, n);
    bool ok = sent == n;

```

```

for (auto [u, id] : mandatory)
    ok &= mcf.g[u][id].cap == 0;
if (!ok)
    cout << "IMPOSSIBLE\n";
else
    cout << -cost - big * required << '\n';
}

```

7.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add_edge(self, u, v, cap, cost):
        ref = (u, len(self.g[u]))
        self.g[u].append([v, cap, cost, len(self.g[v]), cap])
        self.g[v].append([u, 0, -cost, len(self.g[u]) - 1, 0])
        return ref

    def flow(self, s, t, limit):
        n = len(self.g)
        inf = 10**40
        potential = [inf] * n
        potential[s] = 0
        for _ in range(n):
            changed = False
            for u in range(n):
                if potential[u] == inf:
                    continue
                for v, cap, cost, _, _ in self.g[u]:
                    if cap and potential[v] > potential[u] + cost:
                        potential[v] = potential[u] + cost
                        changed = True
            if not changed:
                break
        potential = [0 if x == inf else x for x in potential]
        sent = total = 0
        while sent < limit:
            dist = [inf] * n
            prev = [None] * n
            dist[s] = 0
            pq = [(0, s)]
            while pq:

```

```

        d, u = heapq.heappop(pq)
        if d != dist[u]:
            continue
        for i, (v, cap, cost, _, _) in enumerate(self.g[u]):
            nd = d + cost + potential[u] - potential[v]
            if cap and nd < dist[v]:
                dist[v] = nd
                prev[v] = (u, i)
                heapq.heappush(pq, (nd, v))
    if dist[t] == inf:
        break
    for v in range(n):
        if dist[v] < inf:
            potential[v] += dist[v]
    add = limit - sent
    v = t
    while v != s:
        u, i = prev[v]
        add = min(add, self.g[u][i][1])
        v = u
    v = t
    while v != s:
        u, i = prev[v]
        e = self.g[u][i]
        total += add * e[2]
        e[1] -= add
        self.g[v][e[3]][1] += add
        v = u
    sent += add
    return sent, total

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, p, m = data[:3]
    at = 3
    projects = []
    for _ in range(p):
        projects.append(tuple(data[at : at + 3]))
        at += 3
    choices = []
    max_profit = 0
    for _ in range(m):
        w, j, profit = data[at : at + 3]
        at += 3
        choices.append((w - 1, j - 1, profit))
        max_profit = max(max_profit, profit)
    s, t = n + p, n + p + 1
    mcf = MCF(t + 1)

```

```

for w in range(n):
    mcf.add_edge(s, w, 1, 0)
for w, j, profit in choices:
    mcf.add_edge(w, n + j, 1, -profit)
max_a = max(a for _, _, a in projects)
big = (n + 1) * (max_profit + 2 * max_a * n + 1)
mandatory = []
for j, (low, high, a) in enumerate(projects):
    for k in range(1, high + 1):
        ref = mcf.add_edge(
            n + j, t, 1, a * (2 * k - 1) - (big if k <= low else 0)
        )
        if k <= low:
            mandatory.append(ref)
sent, cost = mcf.flow(s, t, n)
ok = sent == n and all(mcf.g[u][i][1] == 0 for u, i in mandatory)
print(
    -cost - big * sum(low for low, _, _ in projects)
    if ok
    else "IMPOSSIBLE"
)

if __name__ == "__main__":
    main()

```